

OOP Concepts and Databases with Python

Classes, objects, inheritance, polymorphism, encapsulation, and MySQL

■ Read alongside the slides ■ Every slide example worked out in full

How to use this companion document. This document walks beside the lecture deck. The slides give you the headlines; this guide gives you the *why*. Five colour-coded boxes mark five different kinds of help: **blue** for plain-word intuition, **amber** for a real-life analogy, **green** for a fully worked example (every step shown), **red** for traps to avoid, and **purple** for the one line to remember. Read through once before class, then return to it while doing labs.

Contents

1	The big idea: what is Object-Oriented Programming?	2
2	Classes and objects in Python	3
2.1	Defining a class	3
2.2	Three types of methods	4
3	Encapsulation — hiding the details	6
4	Inheritance — reusing and extending classes	8
5	Polymorphism — one name, many forms	11
6	Exception handling — what to do when things go wrong	14
6.1	The try / except / else / finally structure	14
6.2	Common exception types	15
7	Databases with Python — connecting to MySQL	17
7.1	The four steps every database program follows	17
7.2	Installing the connector	17
7.3	CRUD — the four basic database operations	19
	Glossary & symbols	19

1 The big idea: what is Object-Oriented Programming?

Imagine you had to write code for a hospital. There are doctors, patients, rooms, and prescriptions — each with their own data and actions. Writing all of this as a flat list of variables and functions would turn into a mess very fast.

Object-Oriented Programming, or *OOP*, solves this by letting you group related data and actions together in one place, called a *class*. You then create as many copies (called *objects*) of that class as you need.

The intuition

OOP says: **model your program like the real world**. A class is the blueprint. An object is the thing built from that blueprint. One blueprint, many things.

★ Everyday picture

Think of a cookie cutter (*the class*) and the cookies it makes (*the objects*). Every cookie has the same shape, but you can decorate each one differently — different icing, different sprinkles. That decoration is the object's own data, called its *state*. The cutter itself never changes; only the cookies do.

Python supports OOP natively. The four main ideas in OOP are:

- **Encapsulation** — keeping data and methods together and hiding what outside code does not need to see.
- **Inheritance** — a new class can reuse the code of an existing class.
- **Polymorphism** — the same method name can behave differently in different classes.
- **Abstraction** — showing only the essential features and hiding the complexity.

Where this shows up in ML. Almost every machine-learning library is built with OOP. In scikit-learn, `LinearRegression` and `DecisionTreeClassifier` are both classes. You call `.fit()` on each one — same method name, different behaviour. That is OOP in action.

✓ Key takeaway

OOP groups related data and actions into a class. You make objects from that class. The four pillars are encapsulation, inheritance, polymorphism, and abstraction.

2 Classes and objects in Python

A *class* is a blueprint you write once. An *object* is a live copy of that blueprint. When you create an object, Python runs a special method called `__init__` (pronounced "dunder init") to set up its starting data.

2.1 Defining a class

```
class Dog: # define the blueprint
    species = "Canis lupus" # class variable, shared by ALL dogs

    def __init__(self, name, age): # runs when you create a Dog
        self.name = name # instance variable, unique to each dog
        self.age = age

    def bark(self): # instance method
        print(f"{self.name} says: Woof!")
```

The intuition

Class variable vs instance variable. A class variable is written inside the class but *outside* any method. Every object shares it. An instance variable is written inside `__init__` using `self.name = ...`. Each object keeps its own private copy.

★ Everyday picture

Think of a sports team (*class variable*: team name "Tigers") and each player (*instance variable*: jersey number, height). All players share the same team name; each player has their own jersey.

Worked example: creating Dog objects and calling methods

We want two dogs, Buddy and Max. We create them, then make each one bark.

Step 1. Create the first object.

```
buddy = Dog("Buddy", 3)
```

Python calls `__init__("Buddy", 3)`. It sets `buddy.name = "Buddy"` and `buddy.age = 3`.

Step 2. Create the second object.

```
max_dog = Dog("Max", 5)
```

A completely separate object. `max_dog.name = "Max"`.

Step 3. Access the shared class variable.

```
print(Dog.species)      # "Canis lupus"
print(buddy.species)    # also "Canis lupus"
```

Both dogs and the class itself all report the same species.

Step 4. Call the instance method.

```
buddy.bark()      # Buddy says: Woof!
max_dog.bark()    # Max says: Woof!
```

Each dog uses its own `self.name`, so the output differs.

Step 5. Sense-check. Did we need to pass `buddy` into `bark()`? No — Python passes `self` automatically when you write `buddy.bark()`.

2.2 Three types of methods

Python classes can hold three kinds of methods. The table below shows the difference at a glance.

Method type	First parameter	When to use it
Instance method	<code>self</code> (the object)	Read or change the object's own data.
Class method	<code>cls</code> (the class)	Work with class-level data; use <code>@classmethod</code> .
Static method	nothing special	A helper that belongs here logically but needs no data; use <code>@staticmethod</code> .

Worked example: all three method types in one class

We extend the `Dog` class to see all three methods.

Step 1. Write the class with all three.

```
class Dog:
    count = 0 # class variable

    def __init__(self, name):
        self.name = name # instance variable
        Dog.count += 1 # update class variable

    def bark(self): # instance method
        print(f"{self.name}: Woof!")

    @classmethod
    def how_many(cls): # class method
        print(f"Dogs created: {cls.count}")

    @staticmethod
    def is_good_boy(): # static method
        return True
```

Step 2. Create two dogs and call each method.

```
d1 = Dog("Rex")
d2 = Dog("Fido")

d1.bark()          # Rex: Woof!
```

```
Dog.how_many()          # Dogs created: 2  
print(Dog.is_good_boy()) # True
```

Step 3. Sense-check. After two `Dog()` calls, `Dog.count` should be **2**. It is.

Where this shows up in ML. In PyTorch, `nn.Module` uses instance methods (`forward()`) to run a forward pass, class methods to build special architectures, and static methods for utility functions like weight initialization helpers.

✓ **Key takeaway**

Use `self` for instance methods, `@classmethod` for class data, `@staticmethod` for helpers with no object data needed.

3 Encapsulation — hiding the details

Encapsulation (from the Latin word for "put in a capsule") means two things: (1) bundle data and methods together, and (2) protect the data from being changed by outside code in unexpected ways.

The intuition

Think of the data inside an object as being locked in a capsule. Outside code can only touch it through the doors you deliberately open (public methods). You choose what is readable, what is changeable, and what is hidden.

★ Everyday picture

Your bank account is a real-world capsule. The bank holds your balance (data). You cannot reach into the system and type a new number directly. You must use the allowed actions: deposit, withdraw. The bank's rules (methods) keep your balance safe. That is encapsulation.

Python uses naming conventions to signal access levels:

Prefix	Convention	Meaning
<code>name</code>	no underscore	Public — anyone can read or set it.
<code>_name</code>	single underscore	Protected — use inside the class or subclass only (by convention).
<code>__name</code>	double underscore	Private — Python name-mangles it to make outside access harder.

Worked example: a BankAccount class with encapsulation

We build a bank account where the balance can only change through controlled methods.

Step 1. Define the class.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner # public
        self.__balance = balance # private (name-mangled)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited {amount}. New balance
                  : {self.__balance}")

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Not enough funds.")
        else:
```

```
self.__balance -= amount
print(f"Withdrew {amount}. New balance:
      {self.__balance}")

def get_balance(self): # a controlled door to
    read balance
    return self.__balance
```

Step 2. Create an account and make transactions.

```
acc = BankAccount("Alice", 1000)
acc.deposit(500) # Deposited 500. New balance: 1500
acc.withdraw(200) # Withdrew 200. New balance: 1300
print(acc.get_balance()) # 1300
```

Step 3. Try to reach the private variable directly.

```
print(acc.__balance) # AttributeError!
```

Python mangles the name to `_BankAccount__balance`, so direct access raises an error.

Step 4. Sense-check. Starting with 1000, adding 500 gives 1500, subtracting 200 gives **1300**. Correct.

✗ Watch out

- The single underscore `_name` is *only a convention*. Python does not enforce it. Experienced developers respect it; beginners sometimes ignore it by mistake.
- Double-underscore name mangling is not true encryption. You can still access it as `obj._ClassName__attr` if you really need to — but doing so is a red flag in code reviews.
- Do not confuse `__init__` (double underscores on both sides, called a "dunder") with a private variable. Dunders are special Python methods — not private.

Where this shows up in ML. Keras and PyTorch store internal weights as "private" attributes (prefixed with `_`). You are meant to update them only through `.fit()` or the optimizer, never by writing to them directly. Encapsulation keeps the training loop safe.

✓ Key takeaway

Encapsulation bundles data with its methods and hides what outside code should not touch. Use double underscores (`__`) to make a variable private.

4 Inheritance — reusing and extending classes

Inheritance lets a new class start with everything an existing class already has, and then add or change only the parts it needs.

The class you start from is the *parent class* (also called *base class*). The new class that borrows from it is the *child class* (also called *derived class*).

The intuition

A child class gets all the methods and attributes of the parent for free. It can use them as-is, override them to behave differently, or add brand new ones. Think of it as receiving a full toolkit from your parent and then adding your own custom tools.

★ Everyday picture

Think of a generic **Vehicle** blueprint. It has wheels, a speed, and a method `move()`. Now you want a **Car** and a **Bicycle**. Both move, both have wheels — so both inherit **Vehicle**. But a car also has `honk()` and a bicycle has `ring_bell()` — those go in the child classes. You wrote `move()` only once.

Python has five kinds of inheritance. The table below shows them simply.

Type	Structure	Plain meaning
Single	$A \rightarrow B$	One parent, one child.
Multiple	$A, B \rightarrow C$	Child inherits from two or more parents.
Multilevel	$A \rightarrow B \rightarrow C$	Grandparent, parent, grandchild chain.
Hierarchical	$A \rightarrow B$ and $A \rightarrow C$	One parent, many children.
Hybrid	Mix of the above	Combination of two or more types.

Worked example: Vehicle, Car, and ElectricCar — single and multilevel inheritance

We want a base **Vehicle**, a **Car** child, and an **ElectricCar** grandchild.

Step 1. Write the parent class.

```
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display(self):
        print(f"{self.year} {self.make} {self.model}")
```

Step 2. Write the child class. Use `super()` to call the parent's `__init__` — no need to repeat those lines.

```
class Car(Vehicle):
    # Car inherits
```



```

Vehicle
def __init__(self, make, model, year, doors):
    super().__init__(make, model, year) #
        borrow parent setup
    self.doors = doors

def honk(self):
    print(f"{self.model}: Beep beep!")

```

Step 3. Write the grandchild class.

```

class ElectricCar(Car): # ElectricCar
    inherits Car
    def __init__(self, make, model, year, doors,
range_km):
        super().__init__(make, model, year, doors)
        self.range_km = range_km

    def charge(self):
        print(f"Charging {self.model}... range: {
            self.range_km} km")

```

Step 4. Create objects and call methods.

```

my_car = Car("Toyota", "Corolla", 2022, 4)
my_car.display() # 2022 Toyota Corolla <--
                inherited from Vehicle
my_car.honk()    # Corolla: Beep beep!

ev = ElectricCar("Tesla", "Model 3", 2024, 4, 500)
ev.display()     # 2024 Tesla Model 3 <--
                inherited from Vehicle!
ev.honk()        # Model 3: Beep beep! <--
                inherited from Car!
ev.charge()      # Charging Model 3... range:
                500 km

```

Step 5. Sense-check. `ev` has access to `display()` even though it is defined two levels up. The inheritance chain carries it all the way down. ✓

✗ Watch out

- Always call `super().__init__(...)` in the child's `__init__`. If you forget, the parent's attributes never get created and you will get an `AttributeError` later.
- Multiple inheritance can cause the *diamond problem* — two parents define the same method. Python resolves this with the *Method Resolution Order* (MRO). Use `ClassName.__mro__` to see the order Python will search.

Where this shows up in ML. In PyTorch, every neural network you build inherits from

`nn.Module`. You get `.parameters()`, `.to(device)`, and more for free. You only write `__init__` (your layers) and `forward()` (how data flows). The parent handles the rest.

✓ Key takeaway

Inheritance lets a child class reuse the parent's code and extend it. Always use `super()` to call the parent's initialiser.

5 Polymorphism — one name, many forms

The word *polymorphism* comes from Greek: *poly* = many, *morph* = form. In Python it means: the same method name can do different things depending on which class it belongs to.

The intuition

You press the `play` button on a music player, a video player, and a game — one button, three different actions. In code, you call the same method name, and each class decides what happens. You do not need to know which class you are talking to. Just call `play()`.

★ Everyday picture

A chef, a barista, and a baker all "prepare" something. Ask all three to "prepare" and each one does something different — espresso, croissant, pasta. The instruction is the same; the behaviour is unique to each person. That is polymorphism.

Python achieves polymorphism in two main ways:

- **Method overriding** — a child class rewrites a method it inherited from the parent.
- **Duck typing** — if an object has the right method, Python calls it, regardless of its class type.

Worked example: animal sounds — method overriding

We have a parent `Animal` class and three children. Each child overrides the `sound()` method.

Step 1. Write the parent and three children.

```
class Animal:
    def sound(self):
        print("Some generic animal sound")

class Dog(Animal):
    def sound(self):                # overrides parent
        print("Woof!")

class Cat(Animal):
    def sound(self):                # overrides parent
        print("Meow!")

class Cow(Animal):
    def sound(self):                # overrides parent
        print("Moo!")
```

Step 2. Call the same method on a list of objects.

```
animals = [Dog(), Cat(), Cow()]
for animal in animals:
    animal.sound()
# Woof!
# Meow!
# Moo!
```

Step 3. Note what you did NOT need to do. You never checked if `isinstance(animal, Dog)`. Python called the right `sound()` automatically, based on the actual type of each object. That is the power of polymorphism.

Step 4. Sense-check. Three objects, three different outputs, one loop, one method name. ✓

Worked example: operator overloading — the `__add__` dunder

Python lets you define what `+` means for your own class. This is also polymorphism — the same operator, different behaviour.

Step 1. Write a `Point` class.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y +
                      other.y)

    def __str__(self):
        return f"Point({self.x}, {self.y})"
```

Step 2. Add two points with `+`.

```
p1 = Point(2, 3)
p2 = Point(4, 1)
p3 = p1 + p2          # calls p1.__add__(p2)
print(p3)             # Point(6, 4)
```

Step 3. Work out the result by hand. $x: 2+4=6$. $y: 3+1=4$. Result: `Point(6, 4)`. ✓

Where this shows up in ML. In NumPy and PyTorch, `+` on arrays calls their `__add__` method, which does element-wise addition on GPU or CPU. The `forward()` method on every `nn.Module` subclass is pure polymorphism — the same call dispatches to your custom logic.

 Key takeaway

Polymorphism means the same method or operator name works differently for different classes. Override a parent's method in the child to customise its behaviour.

6 Exception handling — what to do when things go wrong

An *exception* is an error that happens while your program is running (at *runtime*). If you do not catch it, Python stops and prints a traceback. *Exception handling* lets you catch the error, deal with it gracefully, and continue or exit cleanly.

The intuition

A program is like a delivery driver. Sometimes the address is wrong (a `KeyError`), sometimes the road is closed (`IOError`). Without handling, the driver just stops and goes home. With handling, the driver has a plan: ring the bell, leave a note, call the office — and continue the round.

★ Everyday picture

Think of a cash machine. You enter a wrong PIN (a `ValueError`). The machine does not crash and switch off. It catches the error, shows "Incorrect PIN", and lets you try again. That is exception handling: catch the problem, respond sensibly, carry on.

6.1 The try / except / else / finally structure

```
try:
    # code that might fail
    result = 10 / 0
except ZeroDivisionError:
    # runs ONLY when that exception is raised
    print("Cannot divide by zero!")
else:
    # runs ONLY when NO exception was raised in try
    print("Result:", result)
finally:
    # runs ALWAYS, exception or not          good for cleanup
    print("Done.")
```

Worked example: safe division with all four clauses

Step 1. Write a safe-divide function.

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: division by zero.")
        return None
    except TypeError:
        print("Error: both inputs must be numbers.")
        return None
    else:
```

```

        print(f"{a} / {b} = {result:.4f}")
        return result
    finally:
        print("safe_divide() finished.\n")

```

Step 2. Test it with valid inputs.

```

safe_divide(10, 4)
# 10 / 4 = 2.5000
# safe_divide() finished.

```

Step 3. Test it with zero divisor.

```

safe_divide(10, 0)
# Error: division by zero.
# safe_divide() finished.

```

Step 4. Test it with wrong type.

```

safe_divide(10, "two")
# Error: both inputs must be numbers.
# safe_divide() finished.

```

Step 5. Sense-check. The finally block ran in every case. The program never crashed. ✓

6.2 Common exception types

Exception	When it happens
<code>ZeroDivisionError</code>	You divided by zero.
<code>TypeError</code>	Wrong type for an operation (e.g. <code>"2" + 2</code>).
<code>ValueError</code>	Right type, wrong value (e.g. <code>int("abc")</code>).
<code>IndexError</code>	List index out of range.
<code>KeyError</code>	Dict key does not exist.
<code>NameError</code>	Variable not defined.
<code>FileNotFoundError</code>	File path does not exist.
<code>AttributeError</code>	Object has no such attribute or method.
<code>ImportError</code>	Module not found.

✗ Watch out

- Never write `except Exception:` or bare `except:` without a plan. It swallows every error silently, including typos in your own code that you need to see.
- Put specific exceptions first. If `ZeroDivisionError` is a subclass of `ArithmeticError`, list `ZeroDivisionError` before `ArithmeticError` or the specific block will never run.
- The `finally` block runs even if your `except` block raises another exception. Use

it for cleanup (close a file, close a database connection) — not for logic.

Where this shows up in ML. Data pipelines catch `FileNotFoundError` when a dataset file is missing, and `ValueError` when a row has the wrong number of columns. Without handling, a single bad row would crash a long training run.

✓ **Key takeaway**

Wrap risky code in `try`. Catch specific exceptions in `except`. Use `else` for success logic and `finally` for cleanup. Always catch the most specific exception first.

7 Databases with Python — connecting to MySQL

Most real applications store data in a *database* — a system that keeps data safe, lets you search it quickly, and handles many users at once. *MySQL* is one of the most popular relational databases. Python can talk to MySQL through a *connector* module.

The intuition

A connector is a translator. Your Python code speaks Python. MySQL speaks SQL (Structured Query Language). The connector sits in the middle and translates each side's language for the other.

★ Everyday picture

Think of calling a restaurant to order food. You (Python) speak English. The kitchen (MySQL) only reads written order tickets (SQL). The waiter (the connector) takes your spoken order and writes it on a ticket. When the food is ready, the waiter brings it back to you. The waiter is `mysql-connector-python`.

7.1 The four steps every database program follows

1. **Connect** — open a channel to the database.
2. **Cursor** — create a cursor, the object that sends SQL.
3. **Execute** — send SQL commands.
4. **Close** — close cursor and connection when done.

7.2 Installing the connector

```
pip install mysql-connector-python
```

Worked example: connect, create a table, insert a row, and read it back

Step 1. Connect to MySQL.

```
import mysql.connector

conn = mysql.connector.connect(
    host      = "localhost",
    user      = "root",
    password  = "your_password",
    database  = "school_db"
)
print("Connected:", conn.is_connected())
# Connected: True
```

The four parameters are: `host` (where the server is), `user`, `password`,

and database (which database to use on the server).

Step 2. Create a cursor.

```
cursor = conn.cursor()
```

The cursor is the object you send SQL through.

Step 3. Create a table.

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id      INT AUTO_INCREMENT PRIMARY KEY,
        name    VARCHAR(50),
        grade   INT
    )
""")
```

CREATE TABLE IF NOT EXISTS means: create it only if it does not already exist, so re-running the script is safe.

Step 4. Insert a row.

```
cursor.execute(
    "INSERT INTO students (name, grade) VALUES (%s, %s)",
    ("Alice", 90)
)
conn.commit()    # save the change permanently
print("Row inserted.")
```

Always call `conn.commit()` after changes. Without it, MySQL rolls the change back when you disconnect.

Step 5. Read the data back.

```
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)
# (1, 'Alice', 90)
```

`fetchall()` returns a list of tuples — one tuple per row.

Step 6. Close everything.

```
cursor.close()
conn.close()
print("Connection closed.")
```

Step 7. Sense-check. Did the insert produce one row? Yes — the output shows exactly one tuple. ✓

7.3 CRUD — the four basic database operations

Letter	Operation	SQL keyword	Python call
C	Create	INSERT INTO	<code>cursor.execute(...); conn.commit()</code>
R	Read	SELECT	<code>cursor.execute(...); cursor.fetchall()</code>
U	Update	UPDATE	<code>cursor.execute(...); conn.commit()</code>
D	Delete	DELETE	<code>cursor.execute(...); conn.commit()</code>

✗ Watch out

- **Never put user input directly into an SQL string.** This is called a *SQL injection attack*. Always use parameterised queries: `cursor.execute("...%s...", (value,))`. The connector escapes the value safely.
- **Always call `conn.commit()`** after INSERT, UPDATE, or DELETE. Without it your changes are invisible to everyone else — and lost when you disconnect.
- **Always close the connection.** Use a `try / finally` block so the connection closes even if an error occurs mid-script.

Where this shows up in ML. Most production ML systems store training metadata, model versions, and experiment results in a database. MLflow, for example, writes run metrics to SQLite or MySQL. Knowing how to query a database is a core data-science skill.

✓ Key takeaway

Connect → cursor → execute → commit → close. Use parameterised queries (`%s`) to stay safe from SQL injection. Always commit after writes.

Glossary & symbols

Key terms.

OOP	Object-Oriented Programming — a way of writing code by grouping data and actions together in classes.
class	A blueprint that defines the structure and behaviour of objects. Written with <code>class Name:</code> .
object	A live copy of a class, created with <code>Name(...)</code> . Each object has its own state.
instance variable	A variable that belongs to one specific object. Set with <code>self.name = value</code> inside <code>__init__</code> .
class variable	A variable shared by all objects of the class. Defined inside the class but outside any method.
<code>__init__</code>	The constructor method. Python runs it automatically when you create a new object.

self	The first parameter of an instance method. It refers to the object that called the method.
encapsulation	Bundling data and methods together and protecting data from direct outside access.
inheritance	A child class receiving the methods and attributes of a parent class.
super()	A function that calls the parent class's method from inside the child class.
polymorphism	The same method name doing different things in different classes.
method overriding	A child class rewriting a method it inherited from the parent.
exception	An error that happens at runtime. If unhandled, it stops the program.
try / except	The block structure for catching and handling exceptions.
finally	A block that runs after try/except regardless of whether an error occurred. Used for cleanup.
connector	A Python module that lets Python talk to a database using SQL.
cursor	An object returned by the connector that you use to send SQL queries.
commit	Saving a database change permanently. <code>conn.commit()</code> .
CRUD	Create, Read, Update, Delete — the four basic database operations.
SQL injection	A security attack where user input is interpreted as SQL. Prevented by parameterised queries.

Symbols and notation at a glance.

Symbol / notation	Meaning
<code>self</code>	The current object inside an instance method.
<code>cls</code>	The class itself inside a class method.
<code>__init__</code>	Constructor dunder — runs on object creation.
<code>_name</code>	Protected attribute (convention only).
<code>__name</code>	Private attribute (name-mangled by Python).
<code>super()</code>	Calls the parent class's version of a method.
<code>%s</code>	Parameterised placeholder in SQL queries.
<code>fetchall()</code>	Returns all rows from the last SELECT query as a list of tuples.